

Updated SIH 26170 Project Execution Plan: AI-Driven Anomaly Detection in Component Burn-In & Screening

Executive Summary & Architectural Shift

The original execution plan split responsibilities awkwardly across the backend, database, and integration layers (specifically between Member 4 and Member 6), while isolating Member 5 strictly to a static UI layout.

This **Updated Execution Plan** streamlines the team architecture by:

- 1. Consolidating **Database + API + System Integration** under a single owner (**Member 5**).
- 2. Shifting the **entire Frontend and UI/UX pipeline** (including HTML, CSS, JavaScript, Chart.js, and WebSocket client logic) to **Member 1**.
- 3. Repurposing **Member 6** as the **Testing, Validation & Demo Engineer** to guarantee system stability and presentation readiness on Day 6.

Revised Team Distribution Matrix

Member	Role	Primary Responsibility
Member 1	Frontend Developer & UI/UX Lead	Complete frontend, dark-mode dashboard, live charts, alerts, controls, and WebSocket client communication.
Member 2	Hardware Simulation & Data Engineer	Generate realistic burn-in telemetry, noise models, thermal drifts, electrical shorts, and baseline CSV/SQL data.
Member 3	Multivariate ML Engineer	Build and tune the Isolation Forest model for sudden chaotic/multivariate anomalies.
Member 4	Time-Series AI Specialist	Build and calibrate the CUSUM stateful algorithm for slow degradation tracking.
Member 5	Database + API + Integration Lead	SQLite database, FastAPI backend, WebSocket server, AI integration, and end-to-end data pipeline bridge.
Member 6	Testing, Validation & Demo Engineer	End-to-end testing, component validation, scenario execution, bug tracking, and presentation/demo preparation.

Detailed Member Work Plans (5-Day Sprints)

Member 1 — Frontend Developer & UI/UX Lead

- **Day 1 (UI Skeleton & Grid):** Build index.html. Establish layout via CSS Grid/Flexbox: Header, central canvas panel for Chart.js, right-side AI FDIR log panel, and scenario control buttons.
- **Day 2 (Theming & Dark Mode):** Write style.css. Implement an ISRO-inspired aerospace dark-mode theme, monospaced typography, and responsive breakpoints.
- **Day 3 (Charts & Components):** Integrate Chart.js via CDN. Initialize multi-axis line charts tracking Temperature, Voltage, and Current.
- **Day 4 (JavaScript & WebSocket Client):** Write app.js. Establish a robust WebSocket client connection to the backend (ws://localhost:8000/ws) to ingest live JSON payloads.
- **Day 5 (Alerts & Controls):** Wire up dynamic DOM manipulation for .alert-red and .alert-yellow FDIR feed updates. Implement event listeners for the Scenario Control Panel buttons.

Member 2 — Hardware Simulation & Data Engineer

- **Day 1 (Component Baseline Logic):** Define variables (V, I, T) and write a loop generating nominal safe state data ($T = 85^{\circ}\text{C} \pm 0.5$).
- **Day 2 (Sensor Noise & Fluctuation):** Inject micro-fluctuations using random.uniform() to simulate real-world sensor jitter.
- **Day 3 (Latent Defect Injection - Drift):** Implement a toggleable thermal runaway function (temperature baseline rises by 0.05°C per second).
- **Day 4 (Catastrophic Failure Injection):** Implement a toggleable electrical short function (Voltage drops instantly while Current spikes massively).
- **Day 5 (Static CSV & SQLite Generation):** Generate 10,000 nominal rows and 1,000 failure rows, saving to sample_data.csv for ML/AI testing.

Member 3 — Multivariate ML Engineer

- **Day 1 (Algorithm Setup & Research):** Import sklearn.ensemble.IsolationForest and study recursive splitting mechanics.
- **Day 2 (Model Training on CSV):** Write a training script parsing Member 2's nominal CSV data to learn baseline healthy states.
- **Day 3 (The Progression Function):** Construct the isolated detection function: def detect_spike(current, voltage, temp):.
- **Day 4 (Output Standardization):** Map model outputs (-1 anomalies) to a clear boolean (True/False) response paired with an anomaly score.
- **Day 5 (Tuning & False Alarms):** Tune the contamination parameter against catastrophic failure data to eliminate false positives on sensor noise.

Member 4 — Time-Series AI Specialist

- **Day 1 (CUSUM Theory & Setup):** Research Cumulative Sum equations, defining parameters for target mean (μ), standard deviation (σ), and sensitivity (k).

- **Day 2 (Stateful Tracking Logic):** Build the DriftDetector Python class maintaining internal cumulative error memory over time.
- **Day 3 (The Detection Function):** Implement core drift calculations ($S^+_n = \max(0, S^+_{n-1} + X_n - (\mu + k))$).
- **Day 4 (Output Standardization):** Write `def evaluate_drift(sensor_value):` to accept single data points, update state memory, and output critical drift flags.
- **Day 5 (Tuning & Calibration):** Run against latent defect datasets to prove drift detection minutes ahead of hard safety thresholds.

Member 5 — Database + API + Integration Lead

- **Day 1 (Database Architecture):** Initialize `burn_in.db` with an SQLite schema containing columns for timestamp, current, voltage, temperature, and anomaly scores.
- **Day 2 (FastAPI Foundation):** Set up `main.py` using FastAPI and Uvicorn. Write standard REST endpoints like `@app.get("/history")`.
- **Day 3 (WebSocket Integration):** Implement the persistent async WebSocket endpoint (`@app.websocket("/ws")`) with connection error-handling blocks.
- **Day 4 (Backend Integration):** Import simulator and AI modules (`simulator.py`, `isolation_forest.py`, `cusum_drift.py`). Pipe simulated telemetry through both AI models.
- **Day 5 (End-to-End Integration):** Unify database insertion loops with live WebSocket JSON broadcasting.

Member 6 — Testing, Validation & Demo Engineer

- **Day 1 (Testing Strategy):** Define validation benchmarks and success criteria across all pipeline layers.
- **Day 2 (Simulator Validation):** Verify data distributions and inspect generated CSV datasets.
- **Day 3 (AI Validation):** Test Isolation Forest and CUSUM accuracy against edge cases and noisy feeds.
- **Day 4 (Pipeline Stress-Test):** Validate SQLite write speeds, FastAPI HTTP latency, and WebSocket resilience under load.
- **Day 5 (Scenario & Demo Packaging):** Script the final hackathon presentation narrative, rehearse live failure injections, and compile final documentation.

Day 6: Master Integration Sequence

On Day 6, the system components converge into a unified runtime loop:

```
\text{[Member 2: simulator.py]} \longrightarrow \text{[Member 5: Backend Bridge]}
\longrightarrow \begin{cases} \text{[Member 3: Isolation Forest]} \\ \text{[Member 4: CUSUM Drift]} \end{cases} \Downarrow \text{[SQLite DB Storage]} \longrightarrow \text{[FastAPI / WebSocket Server]} \longrightarrow \text{[Member 1: JS Client \& Chart.js UI]} \longrightarrow \text{[Member 6: Validated Demo]}
```

Live Telemetry Payload Structure (JSON)

```
{
```

```

"timestamp": "2026-03-30T14:22:10Z",
"temperature": 85.4,
"voltage": 5.0,
"current": 1.21,
"isolation_anomaly": false,
"drift_anomaly": false,
"anomaly_score": 0.04
}

```

Interactive Demo Scenarios

1. **Nominal Burn-In:** Dashboard graphs hover in green ranges. FDIR log remains clear.
2. **Latent Thermal Defect:** Presenter triggers "Inject Thermal Drift." Temperature creeps upward. CUSUM detects the subtle trend early, flashing a yellow warning on the UI.
3. **Sudden Micro-Short:** Presenter triggers "Inject Spike." Voltage drops while current spikes. Isolation Forest instantly flags a multivariate anomaly, triggering a red **CRITICAL FAILURE** alert and halting the simulation.

MEMBER ASSIGNMENT SUMMARY	
M1 (Frontend)	What the judges see in the browser
M2 (Simulator)	What generates virtual chamber telemetry
M3 (Isolation)	Sudden anomaly intelligence
M4 (CUSUM)	Slow degradation intelligence
M5 (Backend/DB)	Everything that bridges and stores data
M6 (Validation)	Proof that the entire prototype works seamlessly